Security Issues — STRIDE Threat Analysis

HMC_BackEnd (Sanaad API) & HMC_Gateway — prepared ahead of Microsoft Threat Modeling Tool (TMT) review

Scope	HMC_BackEnd (NestJS / Oracle) and HMC_Gateway (NestJS proxy + JWT validation)
Methodology	Source-level review mapped to STRIDE categories (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege)
Date	2026-08-18
Prepared by	Devin (AI Software Engineer)

Contents

1. Spoofing

1.1 JWT issuer / audience not validated

Severity	High
Location	`HMC_BackEnd/src/core/auth/jwt.strategy.ts:23-34` `HMC_Gateway/src/core/auth/jwt.strategy.ts`

Both strategies verify the token's signature and expiry only. The code comment states this is deliberate: "issuer/audience checks are intentionally relaxed until the auth spec lands." A token signed with the shared secret is accepted regardless of who issued it.

```
super({
  jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
  ignoreExpiration: false,
  secretOrKey: auth.jwtSecret,
  // no issuer / audience check
});
```

> Risk: if the shared JWT_SECRET is ever exposed (log leak, misconfigured repo, shared with a third system), forged tokens are indistinguishable from real ones.

1.2 Weak hardcoded fallback JWT secret

Severity	High
Location	`HMC_BackEnd/src/core/config/env.validation.ts:34` , `configuration.ts:202` `HMC_Gateway/src/core/config/env.validation.ts:20` , `configuration.ts:59`

```
JWT_SECRET: Joi.string().min(8).default('dev-only-secret-change-me')
```

Identical literal fallback on both projects, with only an 8-character minimum enforced — far too short for HS256. If an environment ever boots without `JWT_SECRET` set, both services silently accept this well-known string.

1.3 Non-production bypass disables MPIN verification

Severity	Medium–High
Location	`HMC_BackEnd/src/modules/auth/application/auth.service.ts:35-37` (same pattern in `onboarding.service.ts` , `mpin.service.ts`)

```
this.devBypass = authDisabled || nodeEnv !== 'production';
```

Any environment whose `NODE_ENV` is not the literal string `"production"` (e.g. `staging` , `uat` , `qa`) skips MPIN verification entirely and issues a token for a dev identity.

1.4 MPIN / OTP persistence is unimplemented

Severity	High (pre-production gap)
Location	`.../auth/infrastructure/adapters/mpin-store.stub.repository.ts` , `otp.stub.repository.ts`

Both adapters throw `NotImplementedException` . Today the dev bypass (1.3) is the only path that functions — this must be replaced with real persistence before any environment holding live credentials goes up.

2. Tampering

2.1 JWT signing algorithm not pinned

Severity	Low–Medium
Location	Same files as 1.1

No explicit `algorithms: ['HS256']` restriction is passed to the strategy. passport-jwt currently defaults sensibly, but pinning removes any dependency-upgrade or config-drift risk of algorithm-confusion attacks.

2.2 Gateway proxy path resolution has no allow-list

Severity	Medium
Location	`HMC_Gateway/src/modules/proxy/proxy.service.ts` (`resolveTargetPath`)

```
private resolveTargetPath(originalUrl, gatewayPrefix, backendPrefix) {
  const suffix = originalUrl.startsWith(trimmedGatewayPrefix)
    ? originalUrl.slice(trimmedGatewayPrefix.length)
    : originalUrl;
  return `${trimmedBackendPrefix}${suffix}`;
}
```

The forwarded path is built with a simple prefix-strip, with no normalization or allow-list of expected backend routes before being handed to axios.

> **Already well-mitigated — no action needed:**
>
> - Oracle procedure calls use named bind variables resolved from the data dictionary; no string concatenation into SQL (`base.repository.ts`).
> - Global `ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true })` blocks mass-assignment / unknown fields (`core.module.ts`).

3. Repudiation

3.1 Diagnostics endpoints lack their own authorization guard

Severity	High
Location	`HMC_BackEnd/src/core/database/diagnostics.controller.ts`

`GET /diagnostics/oracle-object` , `GET /diagnostics/oracle-logs` , `GET /diagnostics/oracle-logs/stats` and `DELETE /diagnostics/oracle-logs` carry no explicit `@Roles` guard of their own — they inherit whatever the global guard default is. Worse, `GET /diagnostics/oracle-logs/view` is explicitly marked `@Public()` and relaxes the CSP to serve an interactive HTML table of every Oracle call the backend has made, including SQL text, bind values and correlation IDs, straight from an unauthenticated route.

```
@Public()
@SkipEnvelope()
@Get('oracle-logs/view')
view(): string {
  return ORACLE_LOG_VIEW_HTML;
}
```

The source comment itself flags this: "*gate/remove it before production if the SQL log is sensitive.*"

> **Already well-mitigated:**

- Sensitive bind values (`mpin/password/otp/secret/token`) are redacted with `***` before logging (`oracle.service.ts`).
- Every request carries a correlation ID and is captured by the audit interceptor with username/device/status (`core/http/correlation-id.middleware.ts` , `core/audit/*`).

4. Information Disclosure

4.1 Gateway relays backend responses byte-for-byte

Severity	Medium
Location	`HMC_Gateway/src/modules/proxy/proxy.service.ts` — `res.send(response.data)`

This is a deliberate architecture choice (documented in the project's own working notes), but it means the backend's exception filter is the *only* layer preventing an Oracle error, stack trace or internal path from reaching the public internet. Any regression there is immediately public.

4.2 CORS defaults to wildcard origin combined with credentials

Severity	Medium (High if shipped as-is)
Location	`HMC_BackEnd/src/main.ts:18-21` , `HMC_Gateway/src/main.ts:18-21`

```
app.enableCors({
  origin: appCfg.corsOrigins.includes('*') ? true : appCfg.corsOrigins,
  credentials: true,
});
```

`CORS_ORIGINS=*` is the shipped `.env.example` default on both projects and is translated into `origin: true` (reflect any origin) while `credentials: true` stays on — a classic pattern that enables cross-site credentialed requests if the default reaches a real deployment.

4.3 Swagger UI publicly reachable

Severity	Medium
Location	`main.ts` in both projects — `/docs`

The full route map, request/response shapes and operation IDs for all 71 Sanaad operations are exposed at `/docs` without any gate on the UI itself (only calling an endpoint from within Swagger needs a bearer token).

> **Already well-mitigated:**
>
> - `AllExceptionsFilter` strips ORA codes, SQL text, stack traces and internal messages before any error reaches the client (`all-exceptions.filter.ts`).
> - `helmet()` applied on both apps for baseline security headers.

5. Denial of Service

5.1 Rate limiting only exists on the gateway's auth endpoints

Severity	Medium
Location	`HMC_Gateway/src/modules/auth/auth.controller.ts` (login/OTP/MPIN) — absent from `HMC_BackEnd/src/app.module.ts`

If the backend is ever reachable directly — misrouted firewall rule, internal network access, a misconfigured load balancer — there is no throttling protecting login attempts or any other endpoint.

5.2 No explicit request body size limit

Severity	Medium
Location	`main.ts` in both projects

Neither bootstrap configures an explicit body-size cap (e.g. `express.json({ limit: '1mb' })`); both rely on Nest/Express platform defaults.

> **Already well-mitigated:**
>
> - Oracle pool sizing (`min=2, max=10`) plus `ORACLE_CALL_TIMEOUT_MS` / `ORACLE_QUEUE_TIMEOUT_MS` (both default 25s) bound how long a hung statement can occupy the pool.

6. Elevation of Privilege

> **This is the most concrete, exploitable category found in this review.**

6.1 IDOR — identity taken from query parameters instead of the authenticated token

Severity	High
Location	`HMC_BackEnd/src/modules/profile/interface/profile.controller.ts` (`GET /profile`) `HMC_BackEnd/src/modules/payslip/interface/payslip.controller.ts` (`GET /payslip/periods` , `/count` , `/`) `HMC_BackEnd/src/modules/employee/interface/employee.controller.ts` (`/performance` , `/employment` , `/basic`)

All of the endpoints above resolve *whose* data to return from a query-string value (`username` , `person_id` , `enum`) rather than from `@CurrentUser()` / the verified JWT claims. Verified directly in source, e.g.:

```
@Get()
generate(@Query() q: PayslipQueryDto) {
  return this.service.generate(q.person_id, q.lang, q.payperiod, q.assignmentid);
  // q.person_id is attacker-controlled; never compared to the caller's identity
}
```

Any authenticated user can view another employee's personal profile, payslip, or performance record simply by changing a query parameter. This directly contradicts the pattern already used correctly on write endpoints:

```
updatePersonal(
  @Body() body: UpdatePersonalRequestDto,
  @CurrentUser() user: AuthenticatedUser,   // correct — identity from the token
  @Lang() lang: LangCode,
) {
  return this.service.updatePersonal(body, user, lang);
}
```

> Impact: exposure of PII and financial data (payslips) across employees — the highest-impact finding in this report.

6.2 Supervisor role check without relationship check

Severity	Medium
Location	`HMC_BackEnd/src/modules/employee/interface/employee.controller.ts` — `supervisorViews`

```
@Get('supervisor/views')
@Roles(Role.SUPERVISOR)
supervisorViews(@Query() q: LovUserQueryDto) {
  return this.supervisor.views(q.username, q.lang);
}
```

`@Roles(Role.SUPERVISOR)` confirms the caller *is a* supervisor, but not that the target `username` is one of their direct reports — a horizontal privilege-escalation gap within the role.

6.3 Function-level access guard defined but unused

Severity	Low
Location	`FunctionAccessGuard` / `@RequireFunction` decorator, not referenced by any controller

The JWT payload already carries a `functions` claim and a matching guard exists, but no controller applies it — fine-grained function-level authorization (e.g. "can access LEAVE but not PAYSLIP") is currently unenforced. Not independently exploitable today since role checks provide a coarser backstop, but it's a defense-in-depth gap worth closing alongside 6.1.

> **Already well-mitigated:**
>
> - MPIN hashing uses `scryptSync` with a random salt and `timingSafeEqual` comparison (`mpin.hasher.ts`).
> - `RolesGuard` is applied globally via `APP_GUARD` , so role checks can't be forgotten on a new controller (`core.module.ts`).
> - Write endpoints (`updatePersonal` , dependents CRUD) correctly bind to `@CurrentUser()` rather than trusting client-supplied identity.

7. Priority Remediation Summary

#	Finding	Severity	Recommended fix
1	IDOR on profile / payslip / employee read endpoints	High	Derive the target identity from `@CurrentUser()` (JWT claims), not from query parameters — mirror the pattern already used on write endpoints.
2	Public / unguarded diagnostics endpoints (Oracle logs, SQL, binds)	High	Remove `@Public()` from `oracle-logs/view` and add an explicit role guard to all `/diagnostics/*` routes, or disable the module outside local/dev.
3	Hardcoded weak JWT secret fallback + missing issuer/audience/algorithm checks	High	Remove the default secret (fail fast if unset), require a long random value, and add `issuer` / `audience` / `algorithms: ['HS256']` to the passport-jwt strategy on both services.
4	Non-production auth bypass tied to a string comparison on `NODE_ENV`	Medium–High	Gate the bypass on an explicit, separately-audited flag rather than "not literally production"; complete the MPIN/OTP store implementation before any shared environment goes live.
5	CORS wildcard + credentials, public Swagger UI	Medium	Set explicit allowed origins per environment; place `/docs` behind auth or an internal-only network path in production.
6	No backend-side rate limiting or body size limit; supervisor relationship not checked	Medium	Add a baseline `ThrottlerModule` and explicit body limit to `HMC_BackEnd`; validate the supervisor–employee relationship server-side before returning supervisor views.

Controls already in place and confirmed sound — no action required, and these can be marked as mitigated in the TMT model:

- Parameterized Oracle procedure calls (no SQL string concatenation)
- Global whitelist/forbid-non-whitelisted DTO validation
- Sensitive-field redaction in Oracle call logs and application logs
- Correlation-ID based audit trail
- Client-facing exception filter that strips technical error detail
- `helmet()` security headers
- Oracle connection pool sizing and call/queue timeouts
- `scrypt` + timing-safe MPIN hashing
- Global `RolesGuard`
- Correct use of `@CurrentUser()` on write endpoints